MINISTRY OF EDUCATION
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY
COLLEGE OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY
Department of Computer Science

جامعة الإمام عبدالرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

Imam Abdulrahman Bin Faisal University
College of Computer Science & Information Technology

CS412 Algorithms Analysis and Design

Project Report

Comparative Analysis of Neural Network Architectures

Instructor: Prof. Sultan Alamri

Group 7M02

Student Name	Student ID
Abdulaziz Saqer BinRuqush	[redacted]
Ahmed Amer Al-Shehri	[redacted]
Waleed Khalid Al-Zahrani	[redacted]

Submitted date: 12/04/2025

Table Of Contents

Introduction 3
Related Work 4
Problem Statement 5
Dataset Description 6
1- Dataset 1: MNIST (Simple) 6
2- Dataset 2: Tomato Leaf Disease (Complex) 7
Algorithms Description 8
1- Algorithm 1: Fully Connected Neural Network (FCNN) 8
2- Algorithm 2: CNN 9
Theoretical Analysis 10
1- Definitions 10
2- Algorithm 1: Fully Connected Neural Network (FCNN) 11
3- Algorithm 2: Convolutional Neural Network (CNN) 12
4- Summary Comparison Table 14
Experimental Analysis 15
Conclusion 17
Appendixes 18
References 18

Table Of Figures

Figure 1Visualization of the MNIST Test Dataset 6
Figure 2 Instances of tomato leaf pictures of the PlantVillage dataset. 7
Figure 3 FullyConnectedNN Class implementation 8
Figure 4 SimpleCNN Class implementation 9
Figure 5 Trainable Parameters Comparison (affect Space complexity) 16

Tables

Table 1 Summary Comparison FCNN vs CNN 14
Table 2 Summary Running Time Comparison of FCNN vs CNN 15

Introduction

Artificial Intelligence and Machine Learning have transformed modern computing, with Computer Vision standing out as a critical application enabling systems to interpret visual data. Neural Networks are the frontrunner training method for predictions, which learn patterns from input data (Courville, 2016). Until now, the **Fully Connected Neural Network (FCNN)** was the most common trained architecture; However, FCNN sees all pixels from images as independent value without spatial awareness since it mathematically accounts for all components without differentiating relevancy (Nash, 2015). This ultimately results in a combinatorial explosion of processing power needed with increasing image resolution and adjusted bit-depth (Courville, 2016).

Therefore, researchers have moved on to the **Convolutional Neural Network (CNN)**. CNN takes advantage of its atypical architecture through "convolution" which spans a localized filter across a picture to maintain spatial orientation and reduces processing power needed to engage with localized pixels at any time (Yamashita, 2018). Thus, things like edge detection or textures are recognized at different critical levels making CNN the industrially expected image classification (Yamashita, 2018).

However, in corresponding Computer Science learning through **Algorithm Analysis and Design**, accuracy is not the only demand - as much subsequent computational cost must exist to make such architecture worth it. Engineers must always operate within the confines of tradeoffs to speed of processing and quality of prediction. Therefore, this project operates within this space by finding a point of comparison between FCNN and CNN where other projects have merely trained FCNNs and CNNs to compare their accuracy levels. This project seeks to identify a much more stable comparative analysis beyond simple accuracy by seeking out the asymptotic **Time Complexity (Big O) and Space Complexity** of each structure. This will demonstrate how well each algorithm functions compared to increasing dimensionality of the dataset, and how well they exceed or fail to meet expected consensus when greater standards of limitation exist.

Related Work

A lot of research has been done on the differences between Fully Connected Neural Networks (FCNNs) and Convolutional Neural Networks (CNNs), especially when it comes to the balance between model complexity and accuracy.

MNIST's Complexity and Efficiency The literature indicates that for simple datasets such as MNIST, lightweight models are frequently sufficient. Zhang et al. examined the complexity-accuracy trade-off specifically on the MNIST dataset (Dhingra, 2017). Their research, that was published on arXiv, shows that CNNs are usually better, but that optimizing architecture can lead to big memory savings without losing accuracy (Dhingra, 2017). This is in line with standard tests on the MNIST database, which show that basic compression methods can make simple models very useful for low-resolution digits (MNIST, 2024).

Analysis of Time Complexity While theoretical Big-O notation is the standard, recent open-access research has focused on the "Time Complexity" of deep learning layers in practice. Shah and (Bhavsar, 2022) executed a computational analysis measuring the duration required by particular layers. Their research contends that the depth and filter size of CNNs result in a "hidden" operational cost frequently neglected by purely asymptotic analysis.

Application to Complex Agricultural Data FCNNs are generally considered to be insufficient for high-dimensional data, like finding plant diseases. (Attallah, 2023) used the PlantVillage dataset to show that "compact" CNNs are needed to deal with the complicated textures of tomato leaf diseases. This study, published in Horticulturae, validates that non-spatial architectures do not scale effectively for agricultural image classification (Attallah, 2023). (Guerrero-Ibañez, 2023) corroborated this by attaining over 99% accuracy on tomato leaves utilizing CNNs.

Problem Statement

The central research question posed by this project is the **algorithmic efficiency versus accuracy trade-off** in Image Classification tasks. Given simple, low resolution data (such as the MNIST handwritten digits), a Fully Connected Neural Network (FCNN) can provide a high level of accuracy for relatively low computational effort. Still, when the FCNN architecture is challenged with more complicated and higher resolution and with small differences data, the FCNN architecture faces a "curse of dimensionality" (Nash, 2015). The quadratic creation of connections that requires memory and training efforts does not always correspond with increased accuracy (Courville, 2016). Alternatively, convolutional neural networks (CNN) provide a means of adequate image accommodation for more complicated data but at an entirely different levels of overhead due to the sliding windows and depth of feature maps (Yamashita, 2018). Therefore, the goal is to **empirically and theoretically compare** the two approaches.

Specifically, this project seeks to answer the following questions:

1. What is the difference in Time Complexity (Big O) of FCNN and CNN as the input size scales?
2. At what point does the computational cost of a CNN become justified by its better accuracy compared to an FCNN?
3. What do the practical runs find in comparison to theoretical analysis on both simple (MNIST) and complex datasets?

Solving this problem enables future researchers to better determine which neural architecture makes more sense given available computational power and the complexity of data in need of classification.

Dataset Description

To evaluate the efficiency and scalability of the algorithms for both models, we selected a pair of datasets that represent the two ends of the spectrum: a simplistic low-resolution dataset (MNIST) and a complicated high-resolution dataset (Tomato Leaf) (MNIST, 2024) (b, 2019).

1- Dataset 1: MNIST (Simple)

- **Source:** torchvision.datasets.MNIST (Standard benchmark).
- **Content:** Grayscale images of handwritten digits (0-9).

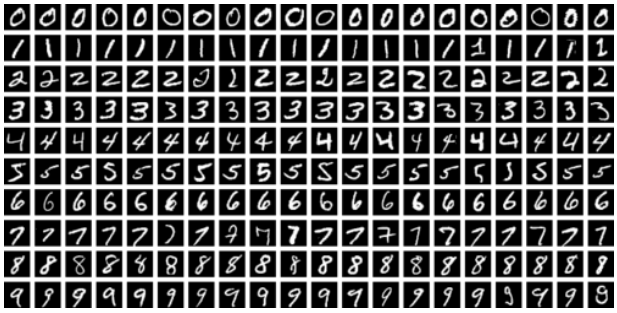

Figure 1Visualization of the MNIST Test Dataset (MNIST, 2024)

Sample images from MNIST test dataset

- **Dimensions:** 28 * 28 pixels (1 Channel).
- **Sample Size:** To manage computational resources and ensure fair comparison, we utilized a subset of the original data:
    - **Training:** 10,000 samples.
    - **Testing:** 2,000 samples.
- **Preprocessing:** Images were converted to tensors and normalized using mean (0.1307,) and standard deviation (0.3081,).
- **Algorithmic Challenge:** Low. The features are simple (strokes and curves), and the input vector size is small (784 pixels), making it solvable by less complex algorithms.

2- Dataset 2: Tomato Leaf Disease (Complex)

- **Source:** Kaggle (kaustubhb999/tomatoleaf).
- **Content:** RGB images of tomato leaves categorized into 10 classes (e.g., Bacterial spot, Late blight, Healthy).

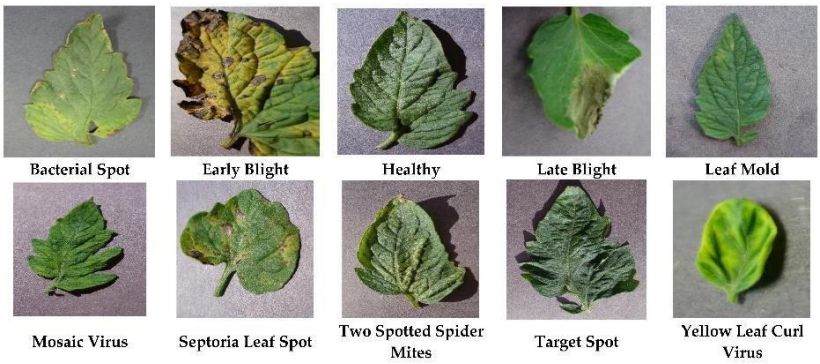

Figure 2 Instances of tomato leaf pictures of the PlantVillage dataset. (Attallah, 2023)

Tomato Leaf Disease

- **Dimensions:** Resized to 256 * 256 pixels (3 Channels: Red, Green, Blue).
- **Sample Size:**
  - **Training:** 10,000 samples.
  - **Testing:** 1,000 samples.
- **Preprocessing:** Images were resized to 256 * 256, converted to tensors, and normalized with mean [0.5, 0.5, 0.5] and std [0.5, 0.5, 0.5].
- **Algorithmic Challenge:** High. The input dimensionality is massive (256 * 256 * 3 = 196,608 input features). The visual patterns (texture of diseases, leaf veins) are complex and require spatial awareness, posing a significant challenge for global processing algorithms like FCNNs.

Algorithms Description

We applied two neural network architectures from different algorithm design families to approach the classification task.

1- Algorithm 1: Fully Connected Neural Network (FCNN)

- **Design Family: Transform and Conquer**. The approach to solving the problem is by transforming the structure of the image in 2D/3D into a 1D vector which is then processed through dense matrix multiplications (Yamashita, 2018).
- **Implementation Class:** (FullyConnectedNN)

```python
class FullyConnectedNN(nn.Module):
    """
    Fully-connected neural network for image classification.
    Complexity: O(N * Din * Dout) where Din = H*W*C (flattened)
    """
    def __init__(self, input_size, num_classes=10, hidden_sizes=None):
        super(FullyConnectedNN, self).__init__()
        self.input_size = input_size
        self.num_classes = num_classes

        if hidden_sizes is None:
            hidden_sizes = [512, 256, 128]

        layers = []
        prev_size = input_size

        for hidden_size in hidden_sizes:
            layers.append(nn.Linear(prev_size, hidden_size))
            layers.append(nn.ReLU(inplace=True))
            layers.append(nn.Dropout(0.5))
            prev_size = hidden_size

        layers.append(nn.Linear(prev_size, num_classes))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        return self.network(x)

print('FullyConnectedNN defined successfully')
```

Figure 3 FullyConnectedNN Class implementation

- **Mechanism:**
    1. **Flattening**: The input image (H * W * C) is flattened into a 1D vector of size N. For the Tomato dataset, this means an input of size 196608.
    2. **Dense Processing**: Each neuron within the layer has a relationship with each neuron within the next layer.
- **Architecture Details:**
    - **Input Layer:** Size is relative to the datasets (28^2=784 for MNIST, 256^2 * 3 = 196k for Tomato).
    - **Hidden Layers:** Three densely connected layers with **512**, **256**, and **128** neurons.
    - **Activation:** ReLU (Rectified Linear Unit) to introduce non-linearity between each of the hidden layers.
    - **Regularization:** Dropout (0.5) to avoid overfitting.
    - **Output:** Linear layer to 10 classes.

2- Algorithm 2: CNN

- **Design Family: Divide and Conquer / Local Search**. The image is taken in smaller receptive fields and processed with common filters (kernels) to derive different features (Nash, 2015).
- **Implementation Class:** (SimpleCNN)

```python
class SimpleCNN(nn.Module):
    """
    Efficient CNN for image classification.
    Complexity: O(N * H * W * K^2 * Cin * Cout)
    """
    def __init__(self, in_channels=1, num_classes=10):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels, 32, kernel_size=3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.relu1 = nn.ReLU(inplace=True)
        self.pool1 = nn.MaxPool2d(2, 2)

        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.relu2 = nn.ReLU(inplace=True)
        self.pool2 = nn.MaxPool2d(2, 2)

        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.relu3 = nn.ReLU(inplace=True)
        self.pool3 = nn.MaxPool2d(2, 2)

        self.adaptive_pool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc1 = nn.Linear(128, 64)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(64, num_classes)
```

```python
    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu1(x)
        x = self.pool1(x)

        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu2(x)
        x = self.pool2(x)

        x = self.conv3(x)
        x = self.bn3(x)
        x = self.relu3(x)
        x = self.pool3(x)

        x = self.adaptive_pool(x)
        x = x.view(x.size(0), -1)
        x = self.fc1(x)
        x = self.dropout(x)
        x = self.fc2(x)
```

Figure 4 SimpleCNN Class implementation

- **Mechanism:**

1. **Feature Extraction:** It attempts to extract edges before shapes and finally textures giving it a sort of hierarchy to the training process (Nash, 2015).
2. **Dimensionality Reduction:** Due to pooling layers, the reduction of the overall informational input size is reduced via downsampling from small fields, aggregating (Nash, 2015).

- **Architecture Components:**
    - **Layer Block 1:** Conv2d (32 filters, 3 * 3 kernel) → BatchNorm → ReLU →MaxPool (2 * 2).

- **Layer Block 2:** Conv2d (64 filters, 3 * 3 kernel) → BatchNorm → ReLU →MaxPool (2 * 2).
- **Layer Block 3:** Conv2d (128 filters, 3 * 3 kernel) → BatchNorm → ReLU →MaxPool (2 * 2).
- **Global Average Pooling:** An AdaptiveAvgPool2d((1, 1)) layer was used to condense the feature maps into a fixed size, regardless of input resolution. This is a critical algorithmic optimization that drastically reduces the parameter count compared to the FCNN (Yamashita, 2018).
- **Classifier**: 2 FC layers (128 neurons → 64 neurons → 10 Output classes) with Dropout.

The largest difference concerns input size. The FCNN gives a separate weight to each and every pixel connection, causing a parameter explosion (O(N)) space complexity where N=pixels). The CNN employs weight sharing (slides the same kernel across the image) and therefore decouples the number of parameters from image size (O(1)) space complexity concerning image size for the conv layers.

Theoretical Analysis

In this section, we analyze the computational complexity of the two implemented algorithms: **Fully Connected Neural Network (FCNN)** and **Convolutional Neural Network (CNN)**. We analyze the **Time Complexity** (number of operations required for a forward pass) and **Space Complexity** (memory required to store trainable parameters) as a function of the input size.

1- Definitions

To formalize the analysis, we define the following variables based on our dataset and code configuration:

- n: Total number of pixels in the input image for a square image of width $W$and height H with C channels:

$$n = W \times H \times C$$

- K: Kernel (filter) size in the CNN (fixed at 3×3; therefore $K = 3$).
- M: Number of neurons in a hidden layer (FCNN) or the number of filters in a CNN layer.

2- Algorithm 1: Fully Connected Neural Network (FCNN)

2.1 Computational Complexity (Time)

The FCNN processes data using dense matrix multiplication. The code implements a dynamic creation of layers based on "hidden_sizes = [512, 256, 128]".

Line-by-Line Analysis:

1. Flattening: "x = x.view(x.size(0), -1)"
    - This transforms the 2D/3D image into a 1D vector of size n. Cost is negligible (memory view change).
2. Input → Hidden Layer 1 : " nn.Linear(prev_size, hidden_size)"
    - Input dimension: n
    - Output dimension: 512
    - Operations: For each of the 512 neurons, we perform n multiplications and additions.
    - Total operations: $512 \times n$
3. Subsequent Hidden Layers:
    - Layer 2: 512×256operations (constant w.r.t. n)
    - Layer 3: 256×128operations (constant)
    - Output: 128×10operations (constant)

Derivation of T(n): The total running time T(n) is the sum of operations across all layers. Let C_{fixed} represent the computations of the hidden layers and output layer, which do not change regardless of input image size.

$$T(n)_{\text{FCNN}} = (512 \cdot n) + C_{\text{fixed}}$$

Order of Growth (Big O): Since 512 is a constant hyperparameter, the dominant term is n.

$$O(n)$$

This indicates that FCNN inference time grows linearly with the number of pixels.

2.2 **Space Complexity (Memory)**

Space complexity is determined by the number of trainable weights (parameters).

- **Input layer parameters**: $n \times 512$
- **Hidden/Output Weights:** Fixed constants (512 * 256 + …)

Space big-O:

$$O(n)$$

While the Big-O is linear, the constant factor is massive. For the Tomato dataset $n = 196{,}608$ the first layer requires ~100M parameters, leading to extreme memory usage.

3- Algorithm 2: Convolutional Neural Network (CNN)

3.1 Computational Complexity (Time)

The CNN processes data using sliding windows (convolutions). The code implements 3 convolutional blocks followed by an Adaptive Average Pool.

Line-by-Line Analysis:

1. Convolutional Layers (self.conv1, self.conv2, self.conv3):
    - The core operation is the dot product of a K×K filter over every pixel in the input map.
    - Let the spatial size of the image For a layer be $H_l \times W_l$:.
    - Let C_{in}be input channels and C_{out} be output filters.
    - Cost per layer : $\text{Cost}_l \approx H_l \cdot W_l \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot K^2$
2. Pooling ("self.pool"): Reduces H and W by factor of 2. Cost is linear to pixels.
3. Adaptive Average Pooling ("self.adaptive_pool"):
    - reduces feature maps to 1×1
    - This decouples the classifier from the input size.

Derivation of $T(n)$

Let $n = H \times W$be the total number of pixels.

$$T(n)_{\text{CNN}} \approx \sum_{l=1}^{3} (n_l \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot K^2)$$

Since $K = 3$ and feature depths (32, 64, 128) are constants, the complexity is proportional to the number of pixels n .

$$T(n)_{\text{CNN}} \approx C_{\text{filters}} \cdot n$$

Order of Growth (Big O):

$$O(n)$$

While the Big O is the same as FCNN (O(n)), the constant factor C_{filters} (representing the kernel operations $K^2 \times$ depth) is significantly larger than the simple multiplication in FCNN. This explains why CNN training takes longer in milliseconds despite having the same asymptotic complexity class.

3.2 Space Complexity (Memory)

This is where the CNN differs fundamentally from the FCNN.

- **Conv Layers:** Weights depend only on filter size (K) and depth, **not** on the input image size n.
  - Layer 1 Weights: 3×3×1×32=288 parameters.
  - Layer 2 Weights: 3×3×32×64=18,432 parameters.
- **Fully Connected Part**: Due to ("AdaptiveAvgPool2d((1, 1))"), the input to the linear layers is always size 128 (the number of filters in conv3), regardless of whether the image is 28×28 or 256×256.

Space big-O:

$$\boldsymbol{O(1)}$$

(With respect to Input Size n). The model size remains constant even if the input image resolution increases to 4K, making CNNs incredibly space-efficient for high-resolution data.

4- Summary Comparison Table

Feature	FCNN (Algorithm 1)	CNN (Algorithm 2)
Core Operation	Matrix Multiplication (Global)	Convolution (Local)
Time Complexity	O(n)	O(n)
Time Constant Factor	Low (Fast per pixel)	High (Slow per pixel)
Space Complexity	O(n)	O(1) (Independent of input size)
Parameter Growth	Explosive. For High-Res (Tomato), params ≈100M.	Constant. For High-Res (Tomato), params ≈100K.
Scalability	Poor. Memory fills up as image size grows.	Excellent. Handles varying resolutions efficiently.

Table 1 Summary Comparison FCNN vs CNN

Experimental Analysis

To validate our theoretical complexity analysis, we conducted a series of experiments comparing the actual performance of the FCNN and CNN on both the simple (MNIST) and complex (Tomato Leaf) datasets.

Quantitative Results

The following table summarizes the consolidated metrics obtained from our code execution:

Dataset	Model	Input Size (n)	Parameters	FLOPs	Avg. Epoch Time (ms)	Final Accuracy
MNIST	CNN	784	102,026	15.1 M	2,706 ms	95.10%
MNIST	FCNN	784	567,434	1.1 M	2,316 ms	94.40%
Tomato	CNN	196,608	102,602	1.34 B	63,874 ms	89.10%
Tomato	FCNN	196,608	100.8 M	201 M	36,979 ms	60.00%

Table 2 Summary Running Time Comparison of FCNN vs CNN

(Note: FLOPs = Floating Point Operations. M = Million, B = Billion)

These findings reinforce the complexity claims made and reveal the specific tradeoffs between the Fully Connected Neural Network (FCNN) and Convolutional Neural Network (CNN) structures. For example, when learning from the low resolution MNIST data set (28 × 28 pixels), the FCNN was computationally faster 2,316ms per epoch (this is the average for 10 epochs) versus 2,706 ms per epoch (this is the average for 10 epochs) for the CNN; this supports theoretical comparison as well because when we have small input vectors (n = 784), the operational overhead of the CNN's sliding window convolutions and pooling layers outweighs the simple, optimized matrix multiplications of the FCNN. Furthermore, no significant differences in accuracy (CNN: 95.1%; FCNN: 94.4%) since these patterns are easy to interpret based on equality digits 0-9, there's no need for dimensionality in CNN operations.

However, this shift in relative efficiency between FCNN and CNN structures becomes clear when learning from more complicated higher resolution Tomato Leaf data set (256 × 256 pixels). First, its computational cost is higher at 63,874 ms per epoch (this is the average for 6 epochs) compared to 36,979 ms per epoch (this is the average for 6 epochs). While the CNN computational runtime is increased , this makes sense relative to its high theoretical FLOP gain requirement (1.34 Billion operations) driven by the massive number of times the shared filter weights must be applied across the larger spatial grid of the input image.

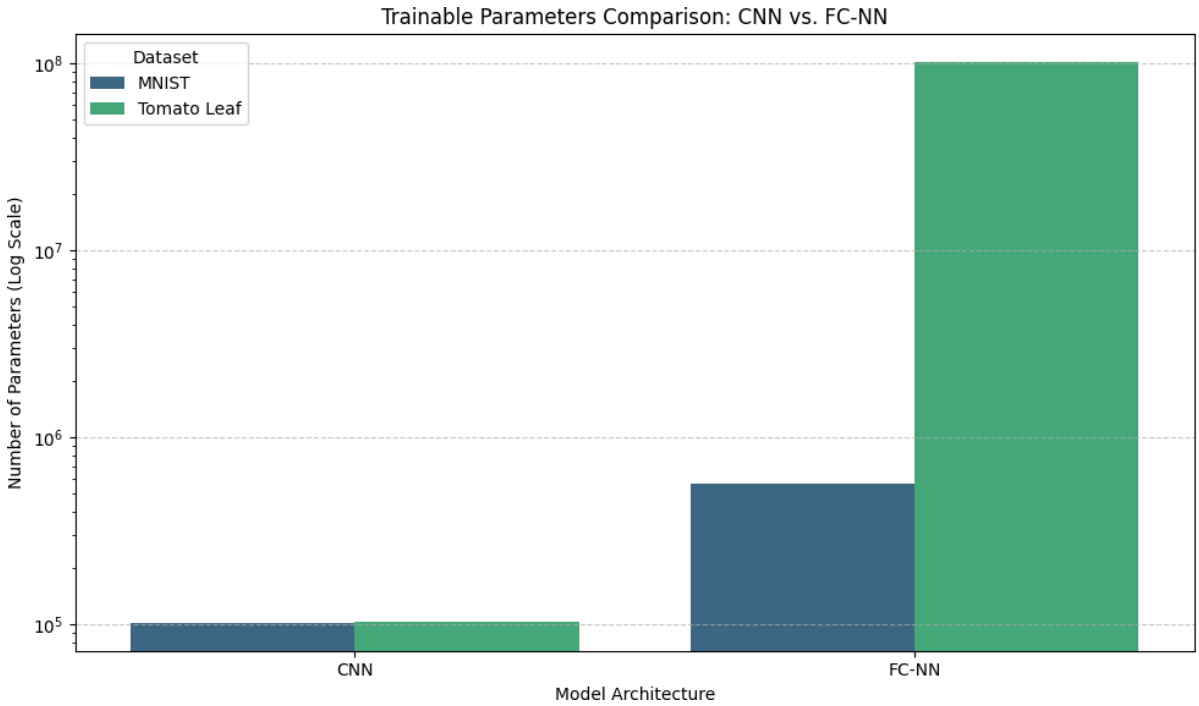

Figure 5 Trainable Parameters Comparison (affect Space complexity)

While CNN was the only algorithmically usable network for this complex problem, FCNN suffered from a catastrophic "parameter explosion" (showed by figure 5) from an existing model of almost 0.5 million parameters required to learn from MNIST but over 100 million parameters required to learn from Tomato dataset. This significant increase in memory use of n = 177× supports the O(n) space complexity found in the theoretical analysis. In addition, the poorly-stable performance of FCNN ultimately resulted in only a 60.0% accuracy - barely better than guessing with this data set - compared to the CNN with a parameter count that averaged a steady (and low) ≈102,000 which achieved a much better accuracy of 89.1%. Thus, while FCNN is faster in calculation per epoch, it's not algorithmically suited for high dimensional images classification, whereas the CNN's higher computational cost is a necessary trade-off for scalable and accurate performance.

Conclusion

This project was a comprehensive comparison of Fully Connected Neural Networks (FCNNs) and Convolutional Neural Networks (CNNs) from algorithmic and empirical standpoints. These two architectures fall under the same O(n) time complexity. the experiments and theoretical evaluations reveal that their practical performance differs depending on the dimensionality and complexity of the input data.

For low-resolution, simple datasets such as MNIST FCNN performed faster per epoch with similar accuracy levels compared to the CNN. The trend makes sense theoretically, when the input size is so low, the costs of the dense matrix multiplication of the FCNN outweighs the overhead introduced by convolution and pooling layers. In such cases, CNN's spatial feature extraction provides only marginal added benefit.

However, the Tomato Leaf dataset highlighted the fundamental limitations of FCNNs. As input size grew (in terms of resolution) the FCNN parameters increased linearly with the number of pixels such that the FCNN became highly inefficient and accuracy declined after it amassed 100 million parameters. In contrast, the CNN parameters increased at virtually a constant parameter count due to weight-sharing of convolutions and due to the locality of convolution operations. Although CNNs incur higher computational cost per epoch, this overhead is justified by their superior ability to extract multi-level spatial features essential for complex image classification.

In summary, FCNNs are useful for small-scale problems when pixel relationships across the image are global and low-dimensional. For any moderate to highly resolution computer vision problem, CNNs are both more efficient and practically favorable. The results validate the theoretical complexities and reinforce the importance of selecting an architecture whose computational design aligns with the dimensionality and structure of the input data.

Appendixes

- Long code therefore attached a Jupyter Notebook for algorithms implementation:

Algorithms_Final (1).ipynb

References

Attallah, O. (2023). Tomato Leaf Disease Classification via Compact Convolutional Neural Networks with Transfer Learning and Feature Selection. *Horticulturae*. Retrieved from https://www.mdpi.com/2311-7524/9/2/149

b, k. (2019). *Tomato leaf disease detection*. Retrieved from kaggle: https://www.kaggle.com/datasets/kaustubhb999/tomatoleaf

Bhavsar, B. S. (2022). Time Complexity in Deep Learning Models. *Procedia Computer Science*, 202-210. Retrieved from https://doi.org/10.1016/j.procs.2022.12.023

Courville, I. G. (2016). *Deep Learning.* Cambridge: MIT Press. Retrieved from https://www.deeplearningbook.org/

Dhingra, A. (2017). Model Complexity-Accuracy Trade-off for a Convolutional Neural Network. *arXiv*. Retrieved from https://arxiv.org/abs/1705.03338

Guerrero-Ibañez, A. a.-M. (2023). Monitoring Tomato Leaf Disease through Convolutional Neural Networks. *Electronics*. Retrieved from https://www.mdpi.com/2079-9292/12/1/229

MNIST. (2024). Retrieved from deeplake: https://datasets.activeloop.ai/docs/ml/datasets/mnist/

Nash, K. O. (2015). An Introduction to Convolutional Neural Networks. *arXiv*.

Yamashita, R. a. (2018). Convolutional neural networks: an overview and application in radiology. *Insights into Imaging*, 611-629. Retrieved from https://doi.org/10.1007/s13244-018-0639-9